ISIL

# Informe de desarrollo
# Proceso de Aprendizaje 3

Módulo de consulta y supervisión del Sistema de Pedidos

**Curso:** Desarrollo de Aplicaciones Empresariales Avanzado

**NRC:** 4622

**Estudiante:** Adrian Antonio Valdez Lozada

**Docente:** [Completar con el nombre indicado en el curso]

**Periodo:** 2026-20

Entrega individual · PA3

# 1. Objetivo y solución desarrollada

El objetivo de esta PA3 es ampliar el Sistema de Pedidos con funciones de consulta y supervisión, aplicando Spring MVC con JSP, una API REST/JSON, controles de Spring Security y consultas explícitas con Spring JDBC. Se conserva el CRUD existente con Spring Data JPA y la integración HTTP con el servicio de inventario.

El alcance funcional de esta iteración no vuelve a implementar las operaciones completas de edición/eliminación ni el flujo de anulación de PA2. Se concentra en consultar los pedidos almacenados y consolidar indicadores por cliente.

### Arquitectura de la solicitud

```
Navegador → Spring Security → DispatcherServlet → Controller JSP / API JSON →
PedidoReporteService → PedidoReporteJdbcRepository → MySQL
PedidoService → PedidoRepository (JPA) para pedidos · InventarioRestClient → inventario-service
para operaciones de inventario
```

La vista JSP y los dos nuevos endpoints JSON llaman al mismo servicio de reportes. La búsqueda avanzada y los agregados se ejecutan contra el historial local; no consultan inventario-service. El diagrama completo de dos niveles se adjunta como archivo Markdown en el proyecto.

### Organización por capa

| Capa | Responsabilidad PA3 | Componentes representativos |
|---|---|---|
| Presentación | Recibir solicitudes, entregar JSP o JSON. | PedidoReporteWebController, PedidoReporteController y reportes-pedidos.jsp. |
| Negocio | Validar parámetros y compartir operaciones de reporte. | PedidoReporteService e implementación. |
| Persistencia | SQL parametrizado y mapeo de resultados; JPA permanece para CRUD. | PedidoReporteJdbcRepository, PedidoRowMapper y proyección agregada. |
| Seguridad | Autenticación, autorización por rol y protección CSRF. | SecurityConfig y filtros de Spring Security. |

El código está en la rama local `feature/pa3-adrian` basada en la rama de trabajo del docente. Aún no se ha creado un commit ni se ha publicado al fork personal.

# 2. Consultas con Spring JDBC

### Búsqueda avanzada

`buscarPedidosPorClienteYTotalMinimo(cliente, totalMinimo)` utiliza `NamedParameterJdbcTemplate` con parámetros separados para el cliente y el monto. La búsqueda parcial no distingue mayúsculas/minúsculas, filtra `total >= totalMinimo`, reutiliza `PedidoRowMapper` y ordena por ID descendente.

### Clientes frecuentes

`listarClientesFrecuentes(minPedidos)` agrupa por cliente y aplica `HAVING COUNT(*) >= :minPedidos`. La proyección retorna:

- **totalPedidos:** cantidad de pedidos incluidos.
- **totalUnidades:** suma de las cantidades.
- **importeAcumulado:** suma del total de pedidos.
- **ticketPromedio:** promedio del total por pedido.

Los valores aportados por el usuario no se concatenan al SQL. La capa Service exige un cliente no vacío, un monto mínimo no negativo y un mínimo de pedidos igual o mayor a uno. Las entradas inválidas se responden como HTTP 400 en la API.

### Convivencia de persistencia

| Mecanismo | Uso | Base de datos |
|---|---|---|
| Spring Data JPA | Operación CRUD transaccional existente de pedidos. | MySQL en ejecución normal; H2 en pruebas. |
| Spring JDBC | Reportes de solo lectura y agregados explícitos requeridos por PA3. | El mismo DataSource MySQL; H2 para pruebas. |
| HTTP RestClient | Consulta/reserva de inventario durante el alta de pedidos. | Servicio Jakarta EE externo con SQL Server. |

Separar las consultas de reporte de la persistencia JPA permite cumplir el requisito de SQL explícito sin sustituir el flujo existente. H2 se activa mediante el perfil de pruebas y permite comprobar las consultas sin MySQL.

# 3. Spring MVC, API y seguridad

### Vista web

`GET /reportes/pedidos` se atiende mediante un controlador Spring MVC que recibe `Model` y selecciona la vista `reportes-pedidos.jsp`. La página presenta el resumen de pedidos, un formulario de cliente y monto mínimo, y un formulario para indicar el umbral de frecuencia. No genera HTML desde el controlador ni duplica reglas de negocio.

### Endpoints JSON

| Método y ruta | Resultado |
|---|---|
| GET /api/v1/pedidos/busqueda-avanzada?cliente=Ana&totalMinimo=100 | Arreglo JSON de pedidos que cumplen ambos filtros. |
| GET /api/v1/pedidos/clientes-frecuentes?minPedidos=2 | Arreglo JSON con los clientes que alcanzan el umbral y sus cuatro métricas. |

### Controles de acceso

- Los roles `OPERADOR` y `CONSULTA` pueden realizar GET a los reportes web y REST.
- El rol `CONSULTA` no puede registrar pedidos con POST a `/pedidos`.
- Un usuario sin sesión es dirigido al formulario de autenticación.
- CSRF permanece habilitado. El POST requiere token válido incluso para `OPERADOR`.

La autenticación es de laboratorio, con usuarios en memoria y claves externalizables. No se agregan JWT, OAuth2 ni persistencia de usuarios, porque no forman parte del requerimiento.

### Continuidad ante inventario no disponible

Las consultas nuevas se apoyan en el historial local de MySQL, por lo que su ruta de ejecución no requiere el cliente HTTP de inventario. La operación de alta sí depende de inventario-service para consultar o reservar el stock. El comportamiento de ambos casos debe verificarse manualmente en el entorno de ejecución.

# 4. Pruebas automatizadas y casos funcionales

### Resultado automatizado verificado

Se ejecutó `mvnw clean test` en el proyecto Spring Boot con el perfil `test`. Resultado de la ejecución: **33 pruebas, 0 fallos, 0 errores, 0 omitidas**. El registro de consola y los reportes Surefire se adjuntan en `entrega-pa3/evidencias/`.

| Prueba | Verificación añadida para PA3 | Resultado |
|---|---|---|
| Repositorio JDBC | Filtro cliente/monto, orden descendente y agregados GROUP BY/HAVING. | Correcta con H2 |
| Servicio de reportes | Rechazo de cliente vacío, monto negativo y umbral menor a uno. | Correcta con Mockito |
| API y MVC | JSON de ambos reportes, HTTP 400 y renderización de la vista. | Correcta con MockMvc/H2 |
| Seguridad | Acceso autorizado al reporte y redirección de usuario anónimo. | Correcta con Spring Security Test |

### Estado de los cinco escenarios manuales

> **Pendiente de ejecución manual:** en esta sesión no estaban disponibles MySQL en localhost:3306, Sistema de Pedidos en localhost:8081 ni inventario-service en localhost:8082. Por ello no se afirma haber obtenido capturas de esos escenarios.

1. Búsqueda por cliente y monto: falta ejecutar y contrastar JSP, API y filas MySQL.
2. Clientes frecuentes: falta validar las métricas contra pedidos de al menos dos clientes en MySQL.
3. Parámetros inválidos: la suite verifica HTTP 400 para monto negativo y mínimo cero; falta la comprobación manual de cliente vacío y los tres valores por interfaz.
4. Inventario indisponible: falta detener el servicio, comprobar fallo controlado del alta y continuidad del reporte local.
5. Roles y CSRF: existen pruebas automatizadas de reporte/anónimo y reglas existentes de POST/CSRF; falta documentar el recorrido manual completo solicitado.

La matriz y los pasos reproducibles están en `entrega-pa3/evidencias/README.md`. Las pruebas automatizadas con H2 no sustituyen las capturas manuales requeridas con MySQL y el servicio externo.

# 5. Conclusiones y referencias

## Conclusiones

1. La aplicación conserva la separación entre controladores, servicio y repositorio, y comparte la lógica de reportes entre JSP y API.
2. Las dos consultas nuevas utilizan Spring JDBC con parámetros nombrados; el reporte agregado usa GROUP BY/HAVING y una proyección tipada.
3. La configuración conserva la autorización por rol y CSRF. La suite automatizada confirma el comportamiento cubierto bajo H2.
4. La rama de trabajo y el commit de implementación están publicados en el fork personal. Queda pendiente ejecutar y documentar los cinco escenarios manuales y completar el nombre del docente en la carátula.

## Repositorio y versión evaluada

Repositorio personal: github.com/Adri123Vldz/isil-2026-ii/tree/feature/pa3-adrian

Rama: `feature/pa3-adrian`

Commit de implementación: 3b2a3edbdadf16d6815e66409bcf407bbcab7e2f

## Referencias


Spring. (s. f.). *JDBC*. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/data-access/jdbc.html

Spring. (s. f.). *Web MVC*. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/web/webmvc.html

Spring. (s. f.). *Authorize HTTP requests*. Spring Security Reference. https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html

Spring. (s. f.). *Cross Site Request Forgery (CSRF)*. Spring Security Reference. https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

Spring. (s. f.). *Testing*. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/reference/testing/index.html


## Archivos de apoyo

- Diagrama: `.github/modernize/assessment/engines/facts/architecture-diagram.md`.
- README, instrucciones de configuración y pruebas: `README.md`.
- Matriz de evidencia y registro automatizado: `entrega-pa3/evidencias/`.

Antes de subir el informe a la plataforma, completar el nombre del docente y adjuntar la evidencia manual real de los cinco escenarios.